Marketing Planner — Technical Manual

Architecture, hosting, security · ICC of Texas

■ Marketing Planner — Technical Manual

Audience: Anyone reviewing the app's architecture — developers, IT admins, or curious non-technical readers

Last updated: 2026-05-14

App version: Production (HEAD ~ commit `d86cf97`+)

Live URL: https://anameen123.github.io/marketing-planner/marketing_schedule_FINAL4.html

> **Two-layer doc.** Each section starts with **the simple explanation** (no jargon) and ends with **the technical detail** for engineers.

Table of contents

13. Trial plan
14. Troubleshooting + monitoring
15. Cost summary

---

1. What the app is

Simple

A web page that the marketing team opens on their phone or laptop to schedule visits, log spending, track lead status, and see what teammates are doing in near real-time.

Technical

- Single-page HTML application (`marketing_schedule_FINAL4.html`, ~1.2 MB self-contained)
- No build step, no framework, no server-side runtime
- All logic runs in the browser as inline JavaScript
- Data persists via Microsoft Graph API to SharePoint files
- Auth via Microsoft Entra ID (MSAL.js v3, SPA + delegated permissions)
- Deployed to GitHub Pages from a public repo

---

2. Where the app lives (hosting)

Simple

The web page is hosted on GitHub Pages — a free service from GitHub (owned by Microsoft). Anyone with the URL can open the page, but only authorized team members can sign in.

Technical

- **Host:** GitHub Pages (free tier)
- **Repo:** https://github.com/anameen123/marketing-planner — public
- **URL:** `https://anameen123.github.io/marketing-planner/marketing_schedule_FINAL4.html`
- **Deploy:** Every push to `main` triggers GitHub's built-in Pages build (~30 sec rebuild)
- **CDN:** GitHub Pages serves via Fastly globally
- **HTTPS:** Enforced automatically
- **No build pipeline** — the HTML file is served as-is

Why public repo?

Free + GitHub Pages doesn't allow private repos with Pages on the free tier. Trade-off:

- Code is publicly readable (anyone can see HTML/JS)
- Data is NOT public (lives in SharePoint, behind Microsoft auth)
- Push protection + pre-commit hook prevents secret leaks
- See SECURITY.md for the policy

---

3. Where the data lives (backend)

Simple

Your data — visits, spending, business info, lead status — all lives inside **your team's SharePoint site** in Microsoft 365. Only the 5 people in the marketing group can see it.

Technical

- **SharePoint site:** `https://wcgtx.sharepoint.com/sites/mkt` (M365 Group: "Marekting team (ON ground+ Digital marketing)")
- **Group ID:** `cdf645b7-5c7e-467d-824c-305c21575e20`
- **Visibility:** Private — only group members + tenant admins have access
- **Storage layer:** SharePoint's default Documents library on that site
- **Folder:** `MarketingPlannerData/` (auto-created on first app bootstrap)
- **Files (JSON):**
- `schedule.json` — all visits, dates, members, spending (~600 KB for full year)
- `business-referrals.json` — referral counts per business + history
- `tier-thresholds.json` — Bronze/Silver/Gold/Platinum thresholds + caps
- `settings-period-budget.json` — global toggle for period-budget enforcement
- `admin-notifications.json` — activity bell entries
- `clinic-leads.json` — lead status (L1-L4) per business
- `display-names.json` — admin's display-name overrides
- **Reports subfolder:** `MarketingPlannerData/Reports/`
- `Monthly/` — auto-exported monthly Excel files (TBD via Power Automate)
- `Quarterly/` — auto-exported quarterly Excel files
- `Manuals/` — this doc + the member manual
- `Archive/` — older snapshots

Why files, not a database?

- File-based storage is simpler than spinning up Cosmos DB or SQL — no admin overhead
- SharePoint files have version history, so accidental overwrites can be rolled back
- Files can be opened/edited by humans directly (e.g., admin can edit `schedule.json` in a pinch)
- Migration to a real DB is straightforward later if scale demands it

---

4. How sign-in works (auth)

Simple

You click "Sign in with Microsoft" → Microsoft asks for your work email + password → if you're authorized, you're in. No separate password to manage; same as Outlook.

Technical

- **Library:** MSAL.js v3 (`@azure/msal-browser@3`), loaded from jsDelivr CDN
- **App registration:** "ICC Marketing Planner" in Entra ID (`wcgtx.com` tenant)
- **Application (client) ID:** `0447f26e-7a21-4c67-90a0-e967ee70a10f`
- **Tenant ID:** `36db94fb-80e4-470d-a675-2ee06ddf3d89`
- **Sign-in audience:** AzureADMyOrg (single tenant — wcgtx.com only)
- **Type:** SPA (Single-page application) — required for MSAL.js
- **Redirect URIs:** 8 variants registered (localhost dev + GitHub Pages prod, with and without .html)
- **API permissions (delegated, Microsoft Graph):**
- `Sites.ReadWrite.All` — read/write SharePoint files on the user's behalf
- `User.Read` — read signed-in user's basic profile (email, name)
- **Admin consent:** Granted by IT (one-time; required because Sites.ReadWrite.All is a high-privilege scope)
- **Auth flow:** Authorization code with PKCE (no client secret, browser-safe)
- **Token caching:** localStorage (survives page reload, ~90-day refresh token lifetime)
- **Email → role mapping:** Hardcoded `EMAIL_USER_MAP` in HTML (will move to SharePoint in future)
- `malthaher@wcgtx.com` → admin
- `dkhan@wcgtx.com` → member (Duaa)
- `skhan@frisco-er.com` → member (Sadia, guest user)
- `aparacha@wcgtx.com` → member (Ahsan)
- `ashuja@wcgtx.com` → readonly (Ahmed, manager)
- **Unauthorized email:** Rejected with explicit error "X is not authorized"

---

5. How data flows (read/write path)

Simple

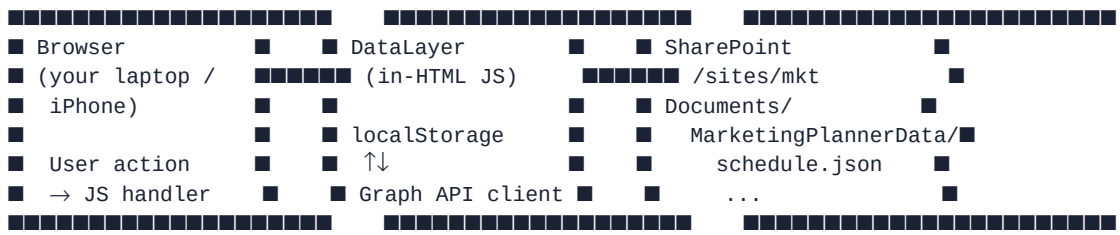
When you click "Save" in the app:

1. Your browser saves the change to its local memory (instant)
2. Within 300ms, the change is uploaded to SharePoint
3. Within 5 seconds, every other team member's browser sees the change

When you open the app:

1. The HTML loads from GitHub Pages
2. You're auto-signed-in (if you've signed in before)
3. The app fetches the latest data from SharePoint
4. Every 5 seconds, the app checks SharePoint for updates from others

Technical



- **Read path** (`DataLayer.load(key)`):
- Returns the value from `localStorage` cache instantly
- 5-sec polling fetches fresh data from SharePoint in background, updates cache, triggers UI refresh
- **Write path** (`DataLayer.save(key, value)`):
- Writes to `localStorage` immediately (optimistic)
- Queues a Graph API write to SharePoint (debounced 300ms)
- On failure: retries on next poll cycle; user sees Sync Error badge
- **Initial bootstrap** (first MS sign-in):
- Acquires Graph API token via MSAL
- Resolves SharePoint site ID + drive ID
- Creates `MarketingPlannerData/` folder if missing
- For each tracked key: if SharePoint has data → pull DOWN; if empty + local has data → seed UP
- Starts polling

6. How live sync works

Simple

Every 5 seconds, your app silently asks SharePoint "anything new?" If yes, your screen updates automatically — no refresh button, no manual sync.

Technical

- **Mechanism:** Polling (not push notifications)
- **Interval:** 5,000 ms (configurable via `DataLayer._config.pollIntervalMs`)
- **What's checked:** Each tracked file in `MarketingPlannerData/`
- **Comparison:** Compare local cache string to fetched content; only re-hydrate UI if different
- **Bandwidth:** ~1 MB per poll for full schedule.json (but compressed) — for a 5-person team, this is negligible
- **Why polling, not push?** Microsoft Graph offers webhooks (push), but they require a public HTTPS endpoint to receive callbacks. Our app has no server — just a browser → SharePoint direct connection. Polling is the simplest pattern that works.

Could it be faster?

- 5 seconds is the default, configurable.
- Lower bound: 1 second (Graph API rate limits prevent <1s polling reliably)
- Pure-instant push: would require a separate Azure Functions endpoint OR a tunneling service. Not justified for a 5-person team.

---

7. What gets saved + where

Simple summary

Type of data	Saved to	Synced to team?
Calendar visits	SharePoint	Yes (5-sec sync)
Spending entries	SharePoint	Yes
Lead statuses	SharePoint	Yes
Referral counts	SharePoint	Yes
Tier thresholds (Bronze/Silver/Gold)	SharePoint	Yes

Type of data	Saved to	Synced to team?
Period budget enforcement toggle	SharePoint	Yes
Admin override notifications	SharePoint	Yes
Display-name overrides	SharePoint	Yes
Per-user "which notifs I read"	localStorage only	No (per-device by design)
Auth tokens (Microsoft session)	localStorage only	No
Browser preferences (UI state)	localStorage only	No

What's NOT saved

- Patient data (none stored)
- Real M365 passwords (Microsoft handles auth; the app never sees them)
- Credit cards or financial accounts (none collected)

---

8. What gets exported + when

Simple

At the end of each month and each quarter, an Excel file gets dropped into the Reports folder in SharePoint. Anyone in the group can open / download it.

Technical

Not yet implemented at the time of this manual — will be set up via Power Automate. Plan:

Frequency	Trigger	What it does
Monthly	Day 1 of each month, 6 AM CT	Reads schedule.json + spending → generates Excel `2026-05.xlsx` → saves to `Reports/Monthly/`
Quarterly	Last day of March / June / September / December, 11 PM CT	Generates `2026-Q1.xlsx` with rollup of tiers, referrals, leads → saves to `Reports/Quarterly/`

Implementation will be a Power Automate flow that reads SharePoint files + uses Office Scripts to format Excel.

Manual CSV export (already in the app)

- The **Reports** tab in the app has a "Bulk Export" section

- Admin clicks → downloads 5-7 CSV files (visits, spending, referrals, leads, etc.) immediately

---

9. What gets reset + how often

Simple

Every 3 months (quarterly), the team's lead-status counts get archived as a snapshot, and the live counts reset to zero. This is so each quarter starts fresh and tiers can be re-ranked.

Technical

- **Trigger:** First app load of a new calendar quarter (Q1 = Jan-Mar, Q2 = Apr-Jun, etc.)
- **What rolls over:**
- Each business's `currentRefs` count is appended to its `history[]` array with the quarter label
- `currentRefs` is reset to 0
- Tier assignments recompute based on new history
- **What doesn't reset:**
- Random + Holiday outreach orgs are **lifetime counters** — they NEVER reset
- All visit history (schedule.json) — visits never deleted
- Lead status (clinic-leads.json) — once L1-L4, stays unless manually changed

Configurable reset frequency (planned)

- Currently quarterly only
- Will add: monthly resets (for trial periods + faster iteration)
- Setting will be `wcg_reset_interval` in `settings-period-budget.json`: `'monthly'` | `'quarterly'`

---

10. Security model

Simple

- Only the 5 people in the Marketing group can see the data
- All sign-ins go through Microsoft (same as Outlook) — no passwords stored in our system
- Code is on GitHub but is automatically scanned to block accidental secret leaks

Technical

- **Access control:** SharePoint group membership (5 people: 1 owner + 3 members + 1 readonly)

- **Auth provider:** Microsoft Entra ID with MFA enforced at tenant level
- **No client secrets in browser:** SPA + PKCE flow; tokens are short-lived (~1 hr) with auto-refresh
- **CORS:** Microsoft Graph allows browser-origin requests with proper bearer token; no proxy needed
- **Secret-scanning:** GitHub Push Protection + gitleaks pre-commit hook
- **Audit trail:** SharePoint logs every file access; Entra ID logs every sign-in
- **Compliance posture:** Same as any M365 SharePoint site — inherited from your tenant's compliance settings
- **Past incidents:** See `SECURITY.md` — 2 secrets leaked + revoked + git history scrubbed on 2026-05-14

What's NOT protected

- A team member sharing their own session by physically handing over their laptop (not unique to our app)
- A team member screenshotting data + emailing it externally (out of scope for tech controls)
- M365 admin / IT can see all data (standard, expected)

---

11. Adding / removing team members

Simple

The admin (Mahmoud) adds new team members in two places:

1. Add their email to the SharePoint group (they get access to data)
2. Add their email to the app's access list (they get the right role)

Technical

- **Step 1: SharePoint group membership** (Microsoft 365 admin / Entra ID)
- Add user to group `cdf645b7-5c7e-467d-824c-305c21575e20`
- Method: M365 admin portal → Groups → "Marketing team..." → Add members
- OR: `az ad group member add --group <id> --member-id <user-id>`
- OR: Graph API POST `/groups/{id}/members/$ref`
- **Step 2: App access list** (currently hardcoded — will be admin UI soon)
- Edit `EMAIL_USER_MAP` in `marketing_schedule_FINAL4.html`
- Add new entry: `'newemail@wcgtx.com': { username: 'newuser', role: 'member', name: 'Full Name', color: '#hex' }`
- Commit + push to GitHub → live in ~30 sec
- **Removing:** Reverse of above. Remove from group + remove from `EMAIL_USER_MAP`.

Planned UX

A future "Team Members" admin panel will let admin add/remove via UI without code edits. Backlog item.

---

12. Data retention + cleanup

Simple

SharePoint keeps everything forever by default. We don't auto-delete anything because:

- Storage cost: zero (well within M365 limits)
- Audit value: history is useful for reports + trend analysis
- Compliance: easier to keep + redact-on-request than to lose

Technical

- **Live data:** `schedule.json` etc. — grow continuously but slowly (~600 KB for 1 year of visits)
- **Historical snapshots:** Quarterly rollover snapshots stored inside `business-referrals.json` (one entry per quarter per business)
- **SharePoint version history:** Every file has automatic version history (~30-90 days of prior versions, depending on tenant setting)
- **No automated deletion:** Manual archive process possible later if data grows past ~10 MB total

If admin ever wants to clean up

- Move old visits (>2 years) to `Reports/Archive/old-visits-2024.json`
- Delete from live `schedule.json`
- Document the cutoff date in SECURITY.md incident log
- Inform team

Server space concern (raised by admin)

- Current data volume: ~600 KB (negligible — M365 gives each user 1 TB+ free)
- No cleanup needed for years at current scale
- If scale grows 100x: revisit + add archival job via Power Automate

---

13. Trial plan

Simple

Test the app for **1 month** with **2 members**. If it works → roll out to the full team. If not → iterate first.

Technical

• **Trial period:** 1 month maximum (configurable via reset frequency setting if you want monthly resets during trial)

• **Test members:** 2 people (admin's choice — likely Duaa + Sadia, since they're the most active)

• **Test scope:**

• Daily use during normal workflow

• Test both desktop + iPhone usage

• Test offline / poor connectivity

• Test the live sync (changes propagating between members)

• Test the monthly Excel export (when set up)

• **Approval criteria:** Test members + admin agree the app:

• Works reliably (no daily bugs)

• Saves time vs. old workflow

• Has all the features they need

• **Approval timeline:** Within 1 week of trial start, decide whether to:

• Continue rollout to full team (Sadia, Ahsan, Manager add)

• Iterate based on feedback (admin lists requested changes, dev cycle resumes)

• Abandon (unlikely but possible)

What to watch during trial

• Sync errors (red badge appearing)

• Slow page loads

• Sign-in difficulties

• Confusing UI for non-tech-savvy members

• Missing features that exist in the old workflow

---

14. Troubleshooting + monitoring

Common issues + fixes

Symptom	Likely cause	Fix
"Can't sign in — admin approval needed"	Tenant requires admin consent for new app	IT clicks "Grant admin consent" in Entra portal (1 click)

Symptom	Likely cause	Fix
"Sync error" red badge persists	SharePoint write failed	Click badge → Reconnect → if persistent, check that user is still in the group
"Not authorized" after sign-in	Email not in EMAIL_USER_MAP	Admin adds the email to the map + redeploys
App is slow	Cold start (first load of day)	Wait 5 sec — subsequent loads are cached
Page shows stale data	Polling missed an update	Hard refresh (Ctrl+F5)
Changes don't appear for teammate	Network issue on their side	Have them refresh + check their sync badge

Monitoring

- **App-side:** Browser console logs (`[DataLayer]` prefix) + sync badge state
- **SharePoint side:** File version history shows when each file was last modified + by whom
- **GitHub side:** Actions tab shows every Pages build + outcome
- **Entra ID side:** Sign-in logs show every authentication attempt (success/fail)

Backups

- SharePoint version history = automatic backup (no admin work needed)
- If a file gets corrupted: admin opens SharePoint → file → Version History → Restore previous version

---

15. Cost summary

Cost	Annual	Why
GitHub Pages hosting	**$0**	Free tier (public repo + Pages)
Microsoft 365 (used for SharePoint + Entra ID)	Already paid	Existing tenant
MSAL.js / Graph API	$0	Free with M365
Power Automate flows	$0	Free with M365 (under flow limits)
Application Insights / monitoring	$0	None used
Total NEW cost for this project	**$0**	All existing infrastructure

Costs we deliberately avoided

- Vercel ($20/user/mo × 5 = $100/mo) — banned by customer + would be $1200/yr
- Azure Static Web Apps ($9/mo + extras) — would require subscription
- Cosmos DB ($5-15/mo) — overkill for 5-user team
- GitHub Pro ($4/mo) for private repo — opted for public repo + security hardening

---

Appendix A — How to deploy code changes

1. Edit `marketing_schedule_FINAL4.html` locally
2. `git add` + `git commit` with a clear message
3. `git push` to main
4. GitHub Pages rebuilds in ~30 sec
5. Live URL serves the new version immediately
6. Users get the new version on their next page load (or after they sign out + back in)

Appendix B — File inventory

```
CODE PROJECT/
■■■ marketing_schedule_FINAL4.html     ← The app (single file, ~1.3 MB)
■■■ logo.jpeg                          ← ICC of Texas logo
■■■ MEMBER_MANUAL.md                   ← User-facing manual (this doc's sibling)
■■■ TECHNICAL_MANUAL.md                ← THIS doc
■■■ SECURITY.md                        ← Security policy + incident log
■■■ README.md                          ← Project overview
■■■ HANDOFF_TO_CLAUDE_CODE.md          ← Detailed project handoff for any maintainer
■■■ AZURE_SHAREPOINT_SETUP.md          ← Old Azure-first plan (back-pocket)
■■■ TEAM_ONBOARDING.md                 ← Short 1-pager for distributing to team
■■■ .gitignore                         ← Excludes backups, node_modules, secrets
■■■ .githooks/pre-commit               ← gitleaks secret-scanner hook
■■■ .github/workflows/                 ← Pages deploy + (defunct) Azure deploy
■■■ (api/, infra/, scripts/, flows/)   ← Azure-path code (kept dormant)
```

Appendix C — Hands-on diagnostic console snippets

For developers debugging issues, the following commands work in browser DevTools console:

```
// Check current user
console.log(CURRENT_USER);

// Check sync state
console.log(DataLayer.mode(), DataLayer._config.lastBootstrapError);

// Force a fresh fetch from SharePoint
```

```
DataLayer._pollOnce_external && DataLayer._pollOnce_external();

// Read a specific stored value
DataLayer.load('wcg_schedule_v1');

// Inspect Microsoft Graph token (debugging auth)
DataLayer._getAccounts_external().then(accs => console.log(accs));
```

End of technical manual. Questions: `malthaher@wcgtx.com`.